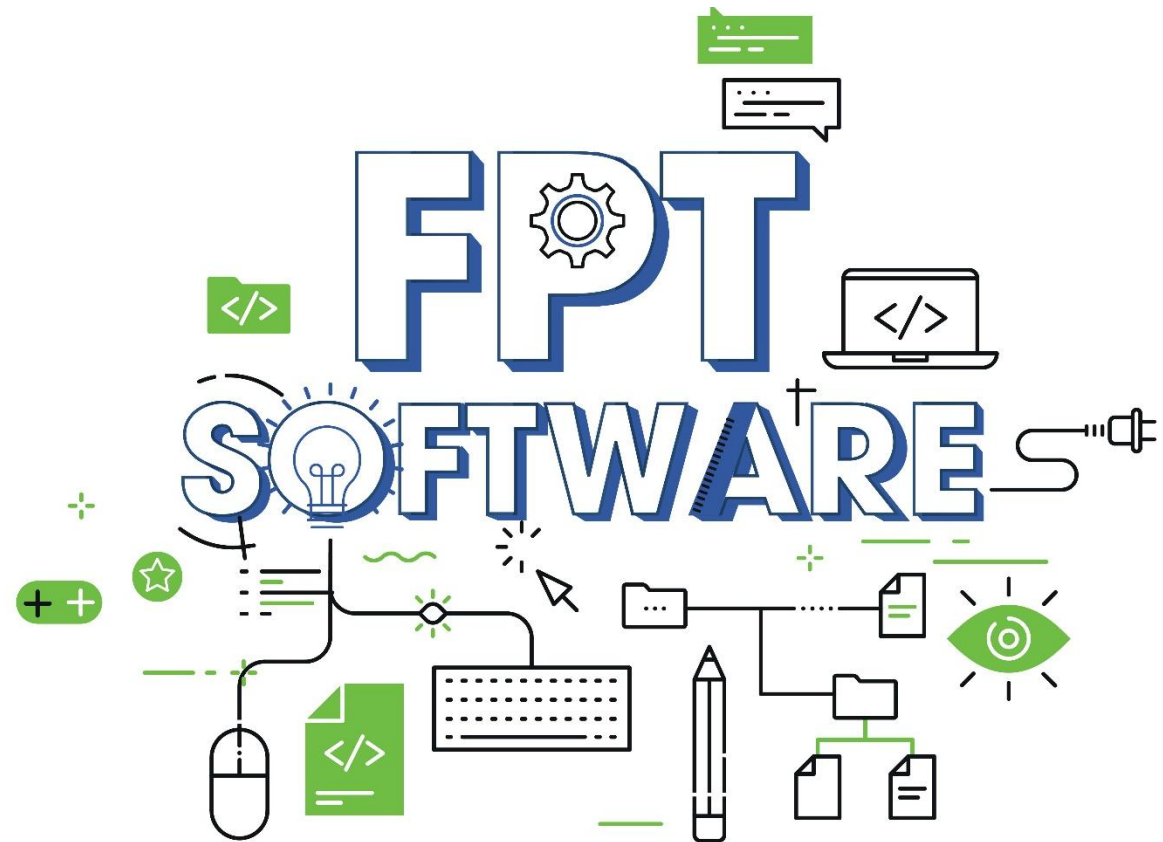


Basic Android

Android Debugging



Agenda

1. What is Debugging?

2. Android Debug Bridge (adb)

3. Debugging Methods in Android



1. What is Debugging?

Concept:

- Debugging is the process of identifying and fixing errors in the source code of an Android application.
- It is an essential part of the development process, helping you identify and rectify issues to ensure that the application operates smoothly and meets user requirements

Objectives:

- Understand how to use debugging tools in Android Studio to locate errors and rectify them.
- Learn common debugging techniques used during the development of Android applications.

2. Android Debug Bridge (adb)

Android Debug Bridge (adb) is a versatile command-line tool that lets you communicate with a device. The adb command facilitates a variety of device actions, such as installing and debugging apps. adb provides access to a Unix shell that you can use to run a variety of commands on a device. It is a client-server program that includes three components:

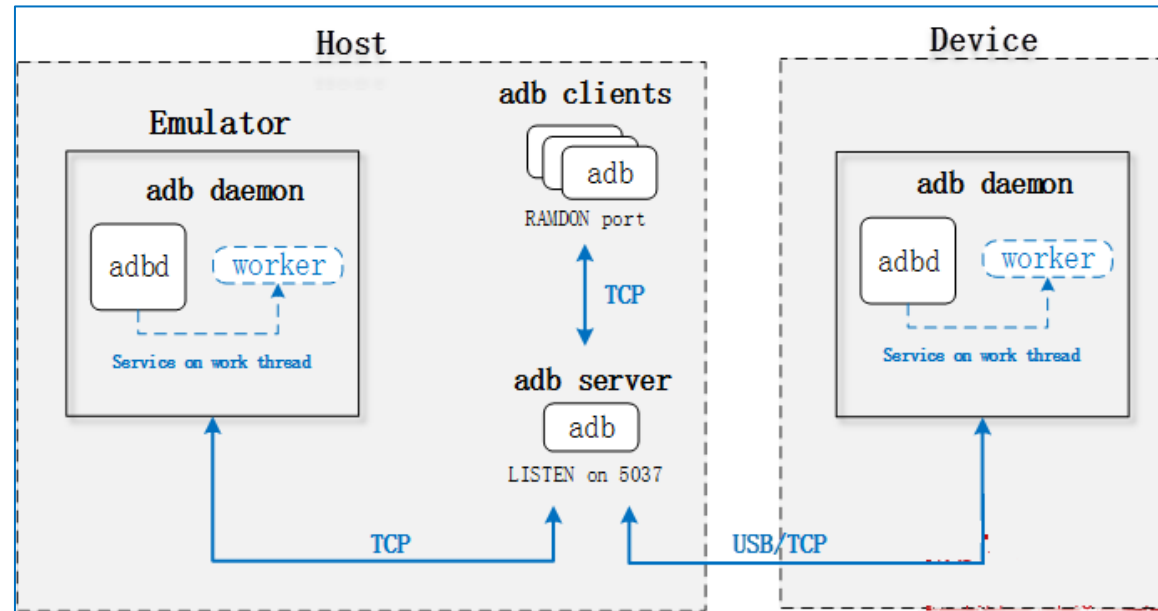
- **A client**, which sends commands. The client runs on your development machine. You can invoke a client from a command-line terminal by issuing an adb command.
- **A daemon (adb)**, which runs commands on a device. The daemon runs as a background process on each device.
- **A server**, which manages communication between the client and the daemon. The server runs as a background process on your development machine.

2. Android Debug Bridge (adb)

How adb works

When you start an adb client, the client first checks whether there is an adb server process already running. If there isn't, it starts the server process. When the server starts, it binds to local TCP port 5037 and listens for commands sent from adb clients.

Note: All **adb** clients use port 5037 to communicate with the **adb** server.



2. Android Debug Bridge (adb)

How adb works

The server then sets up connections to all running devices. It locates emulators by scanning odd-numbered ports in the range 5555 to 5585, which is the range used by the first 16 emulators.

Where the server finds an adb daemon (adbd), it sets up a connection to that port.

Each emulator uses a pair of sequential ports — an even-numbered port for console connections and an odd-numbered port for adb connections. For example:

Emulator 1, console: 5554

Emulator 1, adb: 5555

Emulator 2, console: 5556

Emulator 2, adb: 5557

and so on.

As shown, the emulator connected to adb on port 5555 is the same as the emulator whose console listens on port 5554.

Once the server has set up connections to all devices, you can use adb commands to access those devices. Because the server manages connections to devices and handles commands from multiple adb clients, you can control any device from any client or from a script.

2. Android Debug Bridge (adb)

How to Connect to an device

Pre-Requisite- SDK (Software Development Kit) should be installed on the machine. ADB in Android is packaged with Google's Android SDK (Software Development Kit).

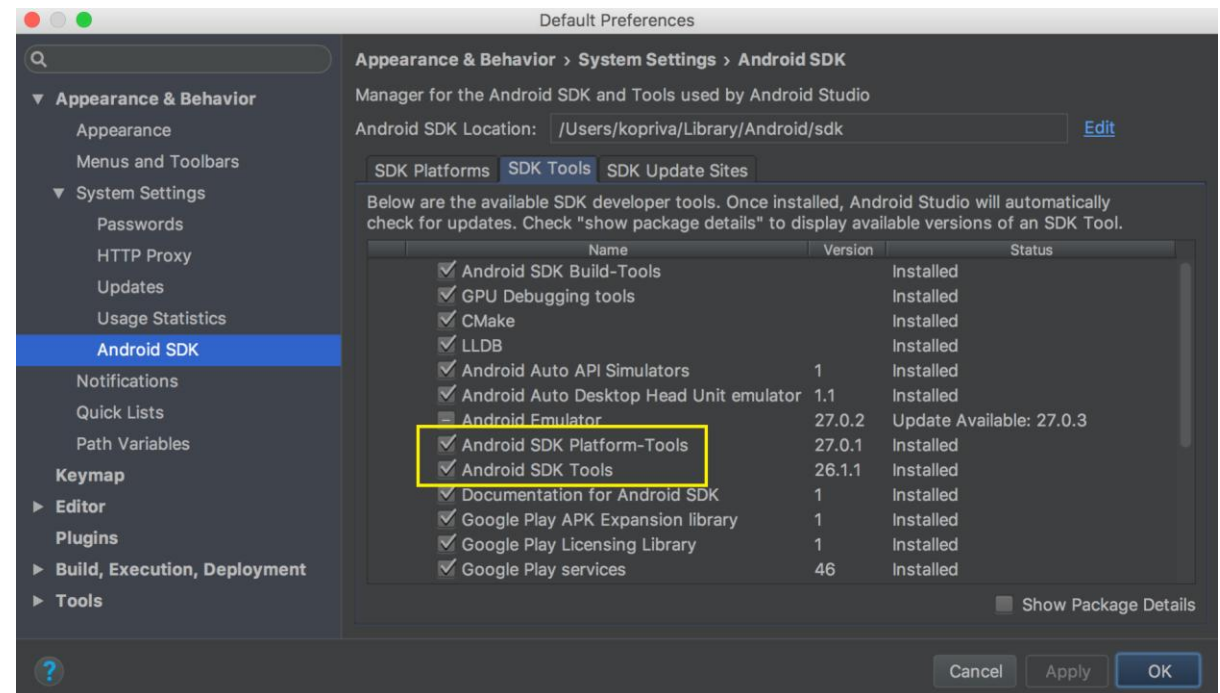
Steps to enable ADB from SDK Manager.

Step 1) Open Android SDK folder

Step 2) Double click on SDK Manager

Step 3) From the list of all packages select Tools and mark the checkbox for

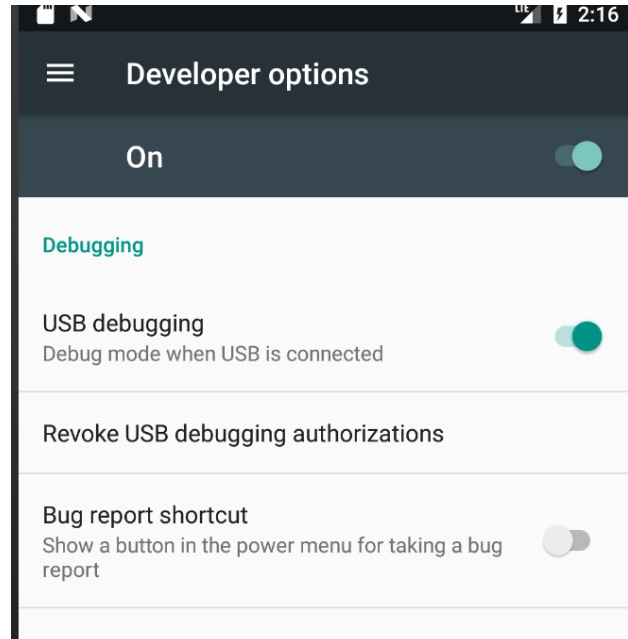
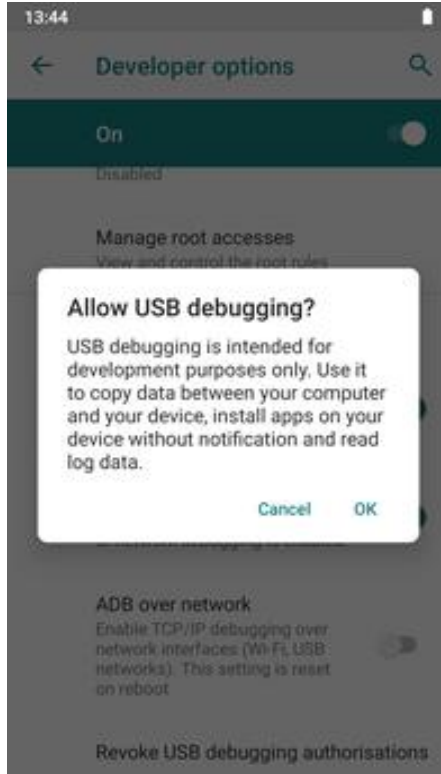
- **Android SDK Tools and**
- **Android SDK Platform-tools.**



2. Android Debug Bridge (adb)

How to Connect Android Device with ADB (Android Debug Bridge) Step 1) Enable USB Debugging on your device

Enable USB debugging option from 'Developer Option' in Android phone. Now, connect your Android device to the computer using USB cable.

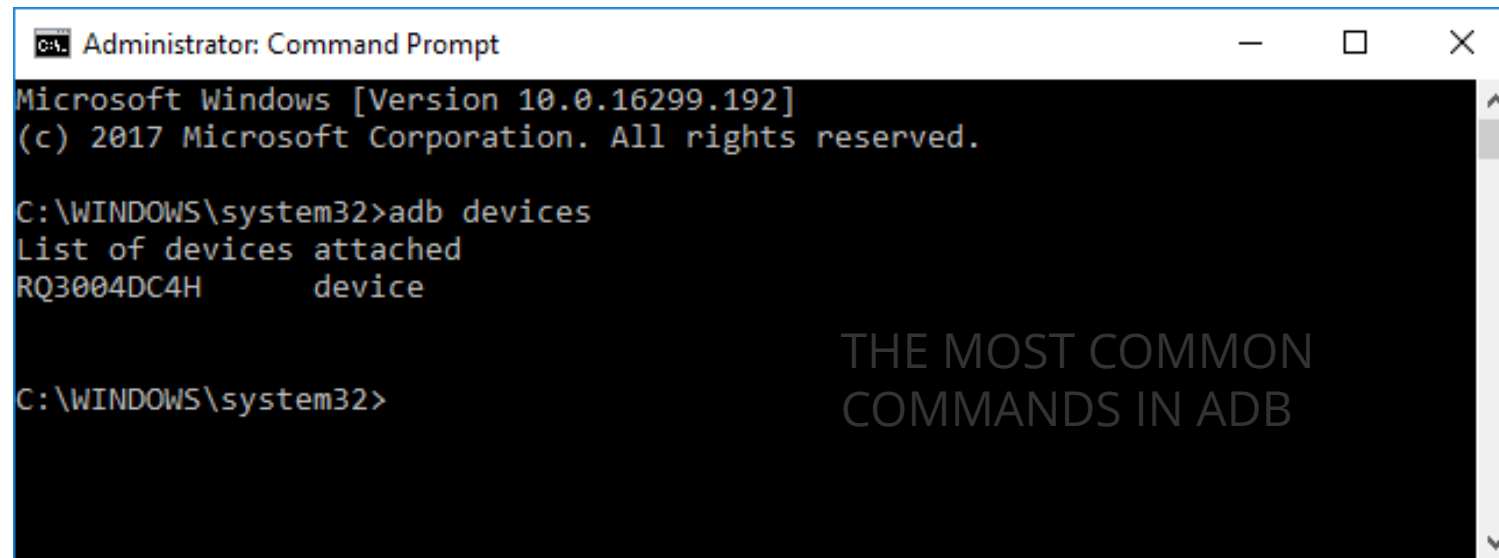


2. Android Debug Bridge (adb)

How to Connect Android Device with ADB (Android Debug Bridge)

Step 2) Open the Command window and Connect an external Android device

This command window will directly open the folder in command prompt window.



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.16299.192]
(c) 2017 Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>adb devices
List of devices attached
RQ3004DC4H      device

C:\WINDOWS\system32>
```

THE MOST COMMON
COMMANDS IN ADB

2. Android Debug Bridge (adb)

THE MOST COMMON COMMANDS IN ADB

Command	Deception
adb	Command output into windows console
adb devices	Displays information about devices connected to the PC and their serial numbers
adb reboot	Android reboot
adb logcat	Compiles and shows logs on the device. The logs are displayed in a command prompt
adb push C:\Users\user\Documents\temp.txt /sdcard	Allows transferring files from PC to mobile device. For this, you have to write address for moving file and a place where it'll be stored on the phone
adb logcat -d > ~/test.log	Compiles logs from the phone and records them in another file
adb shell screenrecord /sdcard/test.mp4	Records a video from the screen of a mobile device mentioning the file location, name, and extension
adb install C:\test.apk	Installs a program on a device. You just have to write a path to the utility
adb shell screencap /sdcard/test.png	Takes a screenshot of the mobile device mentioning the file location, name, and extension

3. Debugging Methods in Android

- **Logging (Logcat)**
- **Breakpoints**
- **Variable Inspection and Viewing Stack Trace**
- **Check Profile your app performance**

3. Debugging Methods in Android

3.1 Logging (Logcat):

- Introduction to Logcat: Logcat is a tool in Android Studio that allows you to view log messages (e.g., Log.d, Log.e, ...) from your application or from apps running on the device and from system, aiding in tracking variables, errors, and program execution flow.

If you want to use the system log to debug your app, make sure your code writes log messages.

- Using Logcat: Guide on how to print logs and utilize different log levels to monitor the application's execution steps.

3.1 Logging (Logcat)

Log levels:

In Android, there are five different log levels that you can use to print log messages using functions like **Log.d()**, **Log.e()**, **Log.i()**, **Log.v()**, and **Log.w()** in your application's source code.

Here's a detailed description of each log level:

Verbose (**Log.v()**):

- The least commonly used log level.
- Used to print the most detailed messages, typically for debugging and monitoring variables, program execution flow.
- Log messages at this level can be numerous and occupy a significant portion of the logcat.

Debug (**Log.d()**):

- Commonly used log level during debugging and application development.
- Used to print messages that aid in tracking the code execution process, checking variable values, and capturing important information.
- Log messages at this level usually appear only when the application is in debug mode.

3.1 Logging (Logcat)

Log levels:

Info (Log.i()):

- Used to record significant information about the application's activities.
- Used to print messages describing the application's state, important events, or to log activity history.
- Log messages at this level may appear during both debug and release modes.

Warning (Log.w()):

- Used to print warnings about potential issues that might lead to errors in the future.
- Used to indicate situations where the application can continue running but exhibits signs that could lead to errors.
- Log messages at this level typically appear during both debug and release modes.

Error (Log.e()):

- Used to print error messages.
- Used to log messages about errors occurring in the application, aiding in tracking and efficiently resolving errors.
- Log messages at this level may appear during both debug and release modes.

3.1 Logging (Logcat)

To view the log messages for your app, do the following.

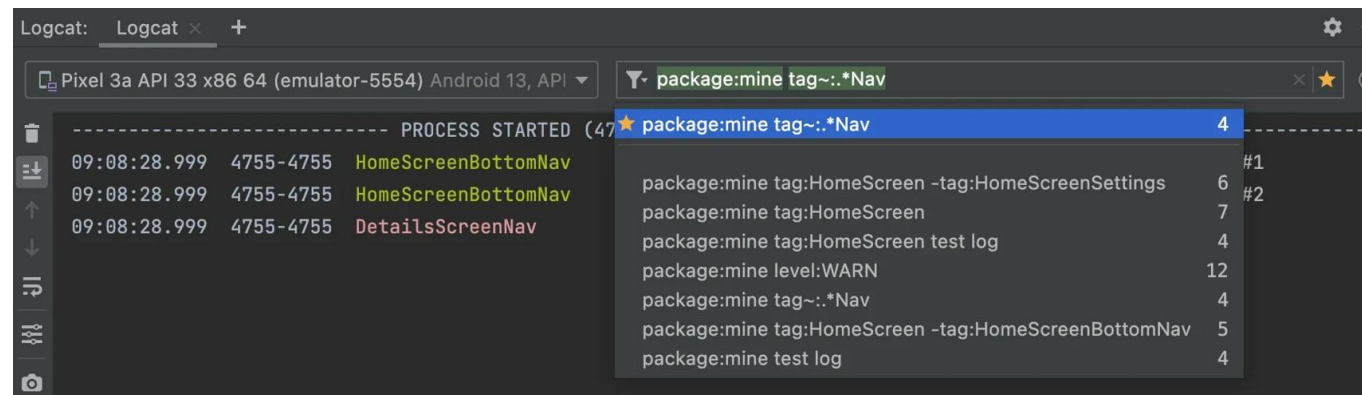
1. In Android Studio, build and run your app on a physical device or emulator.
2. Select View > Tool Windows > Logcat from the menu bar.

```
Logcat: Logcat x +
Pixel XL API 31 (emulator-5554) Android 12, API 31 package:mine
2022-12-29 01:00:55.790 1639-1855 EGL_emulation com.google.samples.apps.sunflower D app_time_stats: avg=27989792.00ms min=8.37ms max=55979576.00ms count=2
2022-12-29 01:01:04.770 1639-1675 .apps.sunflowe com.google.samples.apps.sunflower W Reducing the number of considered missed Gc histogram windows from 5600 to 100
2022-12-29 02:02:23.199 546-603 VerityUtils system_server E Failed to measure fs-verity, errno 1: /data/app/~~arQa19IZQnQgkPbxc1bWag==/com.google.samples.apps.sunflower-gPx95SCti
2022-12-29 02:02:23.400 22470-22470 GraphicsEnvironment com.google.samples.apps.sunflower V ANGLE Developer option for 'com.google.samples.apps.sunflower' set to: 'default'
2022-12-29 02:02:23.400 22470-22470 com.google.samples.apps.sunflower V Neither updatable production driver nor prerelease driver is supported.
2022-12-29 02:02:23.402 22470-22470 NetworkSecurityConfig com.google.samples.apps.sunflower D No Network Security Config specified, using platform default
2022-12-29 02:02:23.402 22470-22470 com.google.samples.apps.sunflower D No Network Security Config specified, using platform default
2022-12-29 02:02:23.621 22470-22584 LibEGL com.google.samples.apps.sunflower D loaded /vendor/lib64/egl/libEGL_emulation.so
2022-12-29 02:02:23.664 22470-22584 com.google.samples.apps.sunflower D loaded /vendor/lib64/egl/libGLESv1_CM_emulation.so
2022-12-29 02:02:23.673 22470-22584 com.google.samples.apps.sunflower D loaded /vendor/lib64/egl/libGLESv2_emulation.so
2022-12-29 02:02:23.743 546-603 VerityUtils system_server E Failed to measure fs-verity, errno 1: /data/app/~~arQa19IZQnQgkPbxc1bWag==/com.google.samples.apps.sunflower-gPx95SCti
2022-12-29 02:02:24.327 22470-22470 .apps.sunflowe com.google.samples.apps.sunflower W Accessing hidden method Landroid/view/View;->computeFitSystemWindows(Landroid/graphics/Rect;Landroid/graphics/Rect;)Z
2022-12-29 02:02:24.328 22470-22470 com.google.samples.apps.sunflower W Accessing hidden method Landroid/view/ViewGroup;->makeOptionalFitsSystemWindows()V (unsupported, reflection, allowed)
2022-12-29 02:02:25.690 22470-22470 Compatibil...geReporter com.google.samples.apps.sunflower D Compat change id reported: 171228096; UID 10148; state: ENABLED
2022-12-29 02:02:26.155 22470-22470 Choreographer com.google.samples.apps.sunflower I Skipped 154 frames! The application may be doing too much work on its main thread.
2022-12-29 02:02:26.579 22470-22582 HostConnection com.google.samples.apps.sunflower D createUnique: call
2022-12-29 02:02:26.584 22470-22582 com.google.samples.apps.sunflower D HostConnection::get() New Host Connection established 0x7df95262cb90, tid 22582
2022-12-29 02:02:26.699 22470-22582 com.google.samples.apps.sunflower D HostComposition ext ANDROID_EMU_CHECKSUM_HELPER_v1 ANDROID_EMU_native_sync_v2 ANDROID_EMU_native_sync_v3 ANDROID_EMU_
2022-12-29 02:02:26.709 22470-22582 OpenGLRenderer com.google.samples.apps.sunflower W Failed to choose config with EGL_SWAP_BEHAVIOR_PRESERVED, retrying without...
2022-12-29 02:02:26.711 22470-22582 com.google.samples.apps.sunflower W Failed to initialize 101010-2 format, error = EGL_SUCCESS
2022-12-29 02:02:26.713 22470-22582 EGL_emulation com.google.samples.apps.sunflower D eglCreateContext: 0x7df95262cf50: maj 3 min 0 rcv 3
2022-12-29 02:02:26.715 22470-22582 com.google.samples.apps.sunflower D eglMakeCurrent: 0x7df95262cf50: ver 3 0 (tinfo 0x7dfb6e942080) (first time)
2022-12-29 02:02:26.803 22470-22582 Gralloc4 com.google.samples.apps.sunflower I mapper 4.x is not supported
```

3.1 Logging (Logcat)

In Android Studio, you can generate key-value searches right from the main query field. This query system provides accuracy of what you want to query and also exclude logs based on key-values.

- ❖ The following are some examples of keys you can use in your query:
 - **tag:** Matches against the tag field of the log entry.
 - **package:** Matches against the package name of the logging app.
 - **process:** Matches against the process name of the logging app.
 - **message:** Matches against the message part of the log entry.
 - **level:** Matches the specified or higher severe log level—for example, DEBUG.
 - **age:** Matches if the entry timestamp is recent. Values are specified as a number followed by a letter specifying the time unit: s for seconds, m for minutes, h for hours and d for days.



3. Debugging Methods in Android

3.2 Breakpoints:

- Breakpoints are points in the code where you set to temporarily pause the program's execution.
- Instructions on setting breakpoints and using them to inspect variable values and track the program's execution flow.

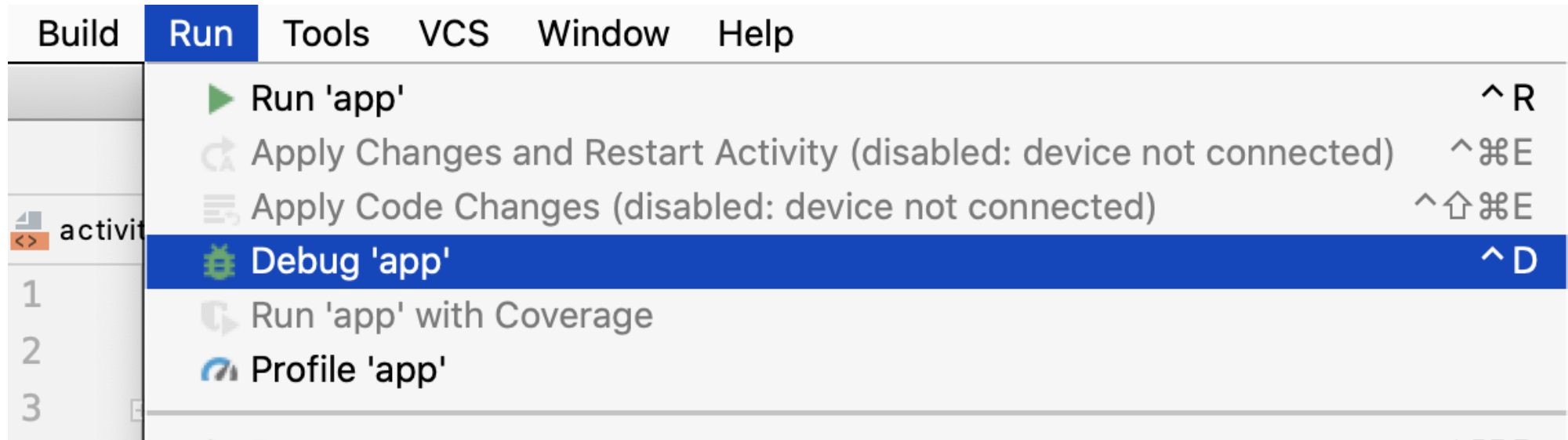
When you learned about logging, you learned to strategically place logs to help identify bugs, and verify that they've been fixed. However, when faced with bugs that you didn't introduce, it's not always clear where to place log statements, or which variables to print out. Often, you can only find this information at runtime.

3.2 Breakpoints

Attach the debugger

Behind the scenes, Android Studio uses a tool called Android Debug Bridge, also known abbreviated as ADB. This is a command-line tool that's integrated into Android Studio, and provides debugging capabilities, such as breakpoints, to your running apps. A tool for debugging is often called a debugger.

To use or attach the debugger to an app, you can't simply run the app with Run > Run like before. Instead, you run the app with Run > Debug 'app'.



3.2 Breakpoints

Add breakpoints to your project

Perform the following steps to see breakpoints in action:

1. Add a breakpoint by clicking the gutter next to the line number you want to pause at. A dot will appear next to the line number, and the line will be highlighted.
2. Run your app with the debugger attached using Run > Debug 'app' or the  icon in the toolbar. When the app launches, you should see a screen like this:

```
13  fun division() {  
14   val numerator = 60  
15  var denominator = 4  
16  repeat( times: 5) { it: Int  
17      println(numerator / denominator)  
18      denominator--  
19  }  
20 }
```

Waiting For Debugger
Application Debugging (process
com.example.debugging) is waiting for
the debugger to attach.

[Force Close](#)

3. Debugging Methods in Android

3.3 Variable Inspection and Viewing Stack Trace: Inspecting Variable Values:

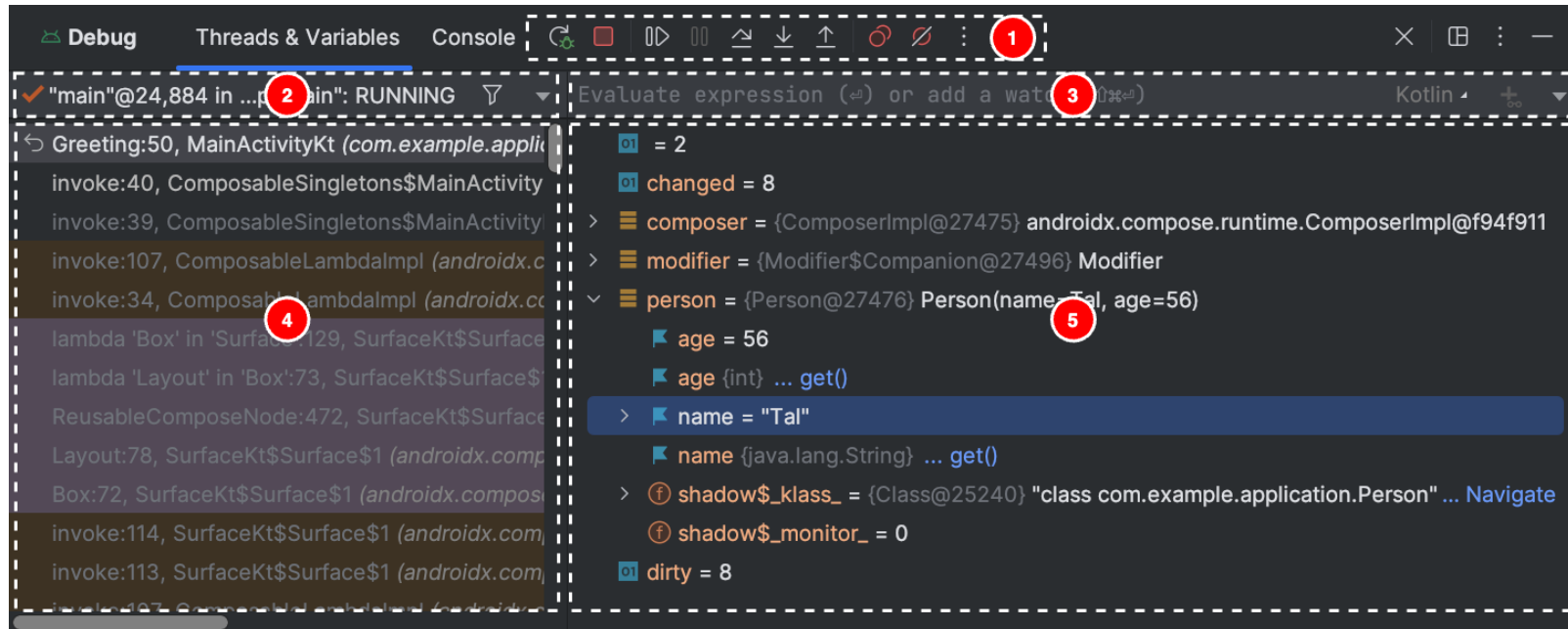
When debugging an Android application, inspecting variable values is a crucial technique to understand how your code is behaving at different points during execution. Android Studio provides tools that allow you to examine the values of variables and their changes in real-time. Here's how to do it:

- **Setting Breakpoints:** Before you can inspect variable values, you usually set breakpoints in your code. Breakpoints are specific lines in your code where you want the execution to pause, allowing you to inspect the variables' values at that point. Once a breakpoint is set and the application execution reaches that point, the debugger will pause, and you can start inspecting the variables.
- **Variable Inspection:** While paused at a breakpoint, you can hover over a variable to see its current value. Android Studio will show you a tooltip with the variable's value. You can also use the "Variables" panel in the Debugger window to view and interact with variables. This panel displays the current state of variables at the breakpoint's location.
- **Watch Expressions:** Android Studio allows you to create watch expressions. These are expressions involving variables that you can monitor during debugging. They can be added to the "Watches" pane in the Debugger window, and their values will be updated each time you step through the code.

3.3 Variable Inspection and Viewing Stack Trace

The debug window

1. Execution and navigation toolbar See Work with breakpoints
2. Thread selector
3. Evaluation and watch expression entry. See Inspect variables.
4. Stack display
5. Variables pane. See Inspect variables.



3.3 Variable Inspection and Viewing Stack Trace

Viewing Stack Trace:

A stack trace is a list of the method calls that have been "stacked" or invoked in the order they occurred. It helps you understand the sequence of function calls leading up to the point where an error occurred. Here's how you can use stack traces for debugging:

- **Exception Stack Traces:**

When an unhandled exception occurs in your code, Android Studio displays an exception stack trace in the "Logcat" panel. This stack trace includes the line numbers and method calls that led to the exception.

By examining the stack trace, you can identify which methods were executing when the error happened.

- **Reading Stack Trace:**

Stack traces are read from top to bottom. The topmost entry is where the error occurred, and the subsequent entries are the methods that were called in sequence leading to that point.

Each entry in the stack trace shows the method name, the class where the method is defined, and the line number.

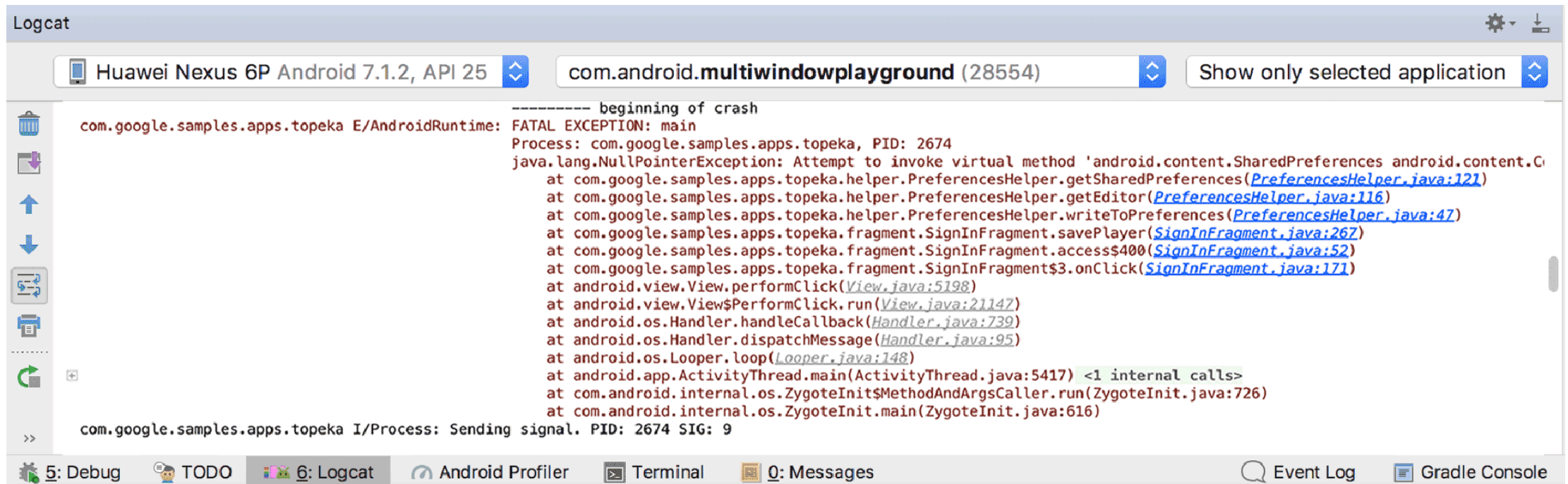
- **Navigating to Source Code:**

In the Logcat or the Debugger panel, you can often click on the method names within the stack trace. Android Studio will take you directly to the corresponding line in the source code, making it easier to locate the problematic code.

3.3 Variable Inspection and Viewing Stack Trace

Exception Stack Traces:

While your app is running in debug mode on a connected device, Android Studio prints and highlights stack traces in the Logcat view, as shown in figure 1. A stack trace shows a list of method calls that lead to the exception being thrown, together with the filenames and line numbers where the calls happened.



3. Debugging Methods in Android

3.4 Check Profile your app performance:

An app is considered to have poor performance if it responds slowly, shows choppy animations, freezes, or consumes too much power. Fixing performance problems involves identifying areas in which your app makes inefficient use of resources such as the CPU, memory, graphics, network, or the device battery.

To find and fix these problems, use the profiling and benchmarking tools and techniques described in this topic. To learn techniques for measuring performance and examples of how to use these techniques to resolve specific problems, see [Measuring performance](#).

Android Studio offers several profiling tools to help find and visualize potential problems:

- **CPU profiler** helps track down runtime performance issues.
- **Memory profiler** helps track memory allocations.
- **Energy profiler** tracks energy usage, which can contribute to battery drain.

3.4 Check Profile your app performance

Inspect CPU activity with CPU Profiler:

Optimizing your app's CPU usage has many advantages, such as providing a faster and smoother user experience and preserving device battery life.

You can use the CPU Profiler to inspect your app's CPU usage and thread activity in real time while interacting with your app, or you can inspect the details in recorded method traces, function traces, and system traces.

The detailed information that the CPU Profiler records and shows is determined by which recording configuration you choose:

- **System Trace:** Captures fine-grained details that allow you to inspect how your app interacts with system resources.
- **Method and function traces:** For each thread in your app process, you can find out which methods (Java) or functions (C/C++) are executed over a period of time, and the CPU resources each method or function consumes during its execution. You can also use method and function traces to identify *callers* and *callees*. A caller is a method or function that invokes another method or function, and a callee is one that is invoked by another method or function. You can use this information to determine which methods or functions are responsible for invoking particular resource-heavy tasks too often and optimize your app's code to avoid unnecessary work.

When recording method traces, you can choose *sampled* or *instrumented* recording. When recording function traces, you can only use sampled recording.

3.4 Check Profile your app performance

Inspect CPU activity with CPU Profiler:

CPU Profiler overview

To open the CPU Profiler, follow these steps:

» Select **View > Tool Windows > Profiler** or click **Profile** in the toolbar.

If prompted by the **Select Deployment Target** dialog, choose the device to which to deploy your app for profiling. If you've connected a device over USB but don't see it listed, ensure that you have [enabled USB debugging](#).

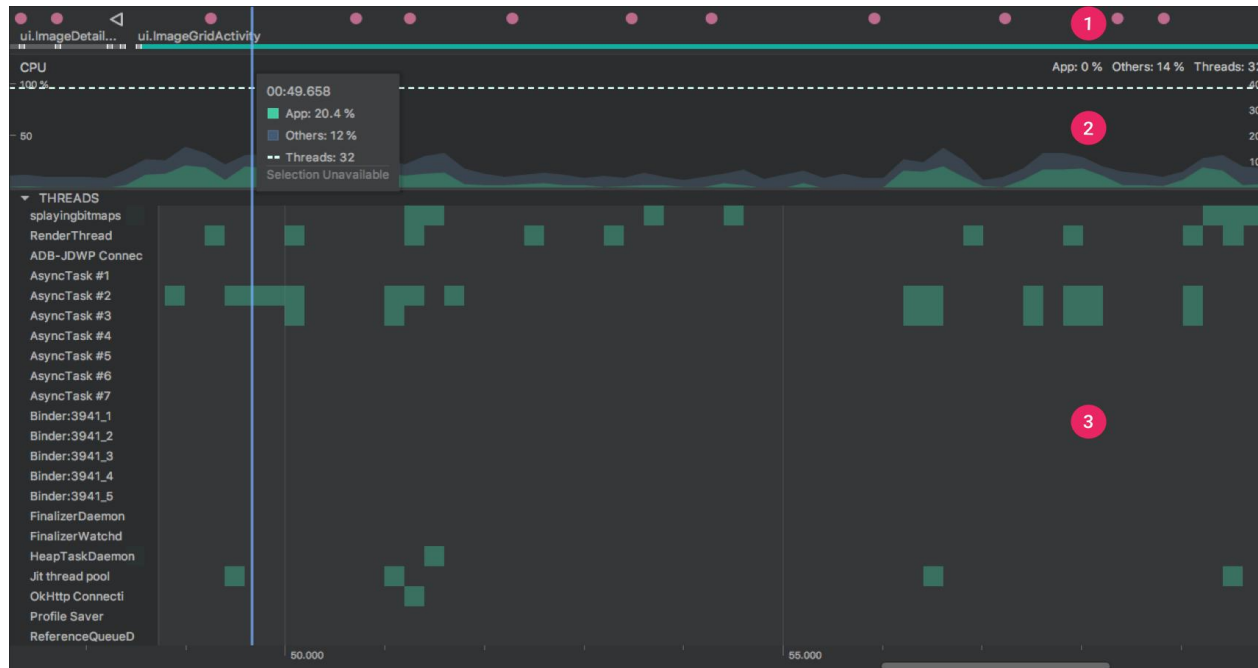
» Click anywhere in the **CPU** timeline to open the CPU Profiler.

3.4 Check Profile your app performance

Inspect CPU activity with CPU Profiler:

CPU Profiler overview

When you open the CPU Profiler, it immediately starts displaying your app's CPU usage and thread activity. You should see something similar to Figure 1.



3.4 Check Profile your app performance

Inspect CPU activity with CPU Profiler:

CPU Profiler overview

1. **Event timeline:** Shows the activities in your app as they transition through different states in their lifecycle, and indicates user interactions with the device, including screen rotation events. For information on enabling the event timeline on devices running Android 7.1 (API level 25) and lower, see [Enable advanced profiling](#).
2. **CPU timeline:** Shows real-time CPU usage of your app—as a percentage of total available CPU time—and the total number of threads your app is using. The timeline also shows the CPU usage of other processes (such as system processes or other apps), so you can compare it to your app's usage. You can inspect historical CPU usage data by moving your mouse along the horizontal axis of the timeline.
3. **Thread activity timeline:** Lists each thread that belongs to your app process and indicates its activity along a timeline using the colors listed below. After you record a trace, you can select a thread from this timeline to inspect its data in the trace pane.
 - **Green:** The thread is active or is ready to use the CPU. That is, it's in a running or runnable state.
 - **Yellow:** The thread is active, but it's waiting on an I/O operation (for example, disk or network I/O) before it can complete its work.
 - **Gray:** The thread is sleeping and is not consuming any CPU time. This sometimes occurs when the thread requires access to a resource that is not yet available. Either the thread goes into voluntary sleep, or the kernel puts the thread to sleep until the required resource becomes available.

The CPU Profiler also reports CPU usage of threads that Android Studio and the Android platform add to your app process—such as JDWP, Profile Saver, Studio:VMStats, Studio:Perfa, and Studio:Heartbeat (although, the exact names displayed in the thread activity timeline may vary). Android Studio reports this data so that you can identify when thread activity and CPU usage are actually caused by your app's code.

3.4 Check Profile your app performance

Inspect your app's memory usage with Memory Profiler:

The Memory Profiler is a component in the [Android Profiler](#) that helps you identify memory leaks and memory churn that can lead to stutter, freezes, and even app crashes. It shows a realtime graph of your app's memory use and lets you capture a heap dump, force garbage collections, and track memory allocations.

To open the Memory Profiler, follow these steps:

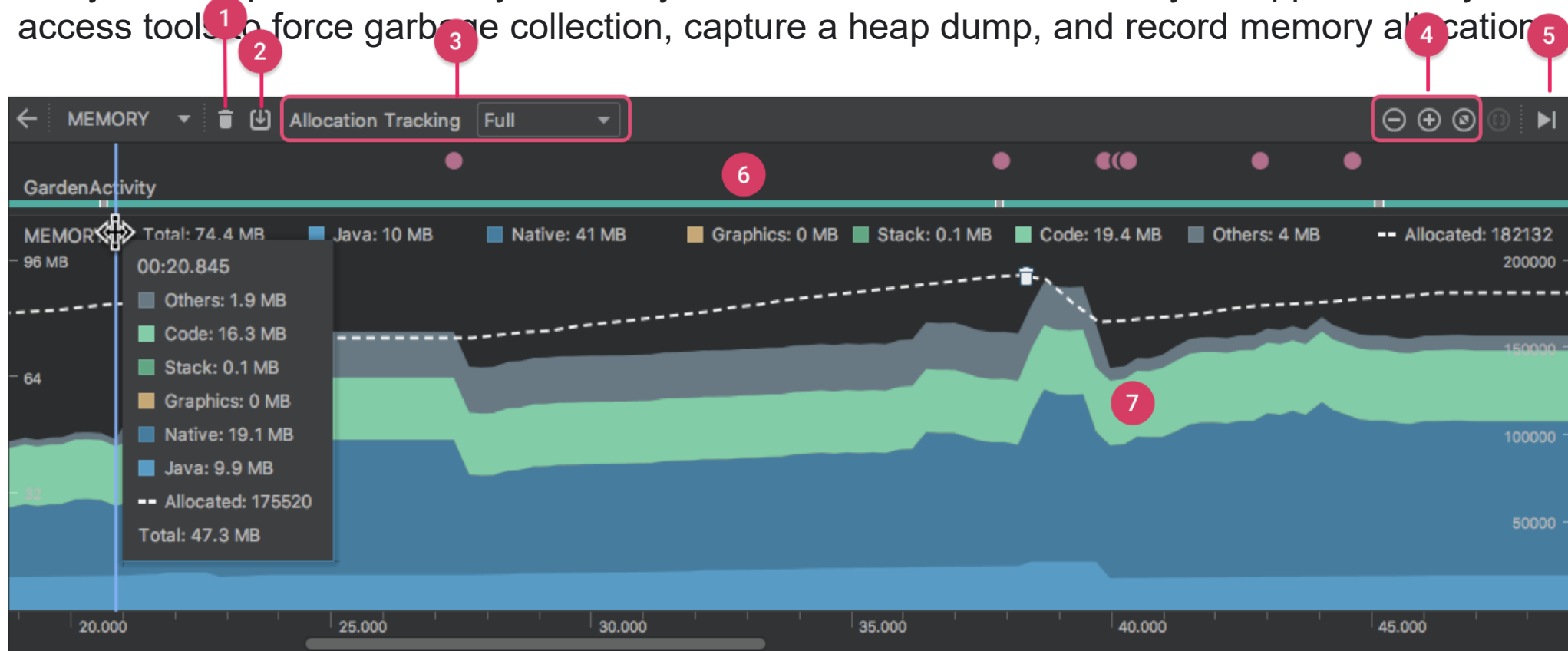
1. Click **View > Tool Windows > Profiler** (you can also click **Profile** in the toolbar).
2. Select the device and app process you want to profile from the Android Profiler toolbar. If you've connected a device over USB but don't see it listed, ensure that you have [enabled USB debugging](#).
3. Click anywhere in the **MEMORY** timeline to open the Memory Profiler.

3.4 Check Profile your app performance

Inspect your app's memory usage with Memory Profiler:

Memory Profiler overview

When you first open the Memory Profiler, you'll see a detailed timeline of your app's memory use and access tools to force garbage collection, capture a heap dump, and record memory allocation.



3.4 Check Profile your app performance

Inspect your app's memory usage with Memory Profiler:

Memory Profiler overview

As indicated in figure 1, the default view for the Memory Profiler includes the following:

- » 1) A button to force a garbage collection event.
- » 2) A button to [capture a heap dump](#).

Note: A button to [record memory allocations](#) appears to the right of the heap dump button only when connected to a device running Android 7.1 (API level 25) or lower.

- » 3) A dropdown menu to specify how frequently the profiler captures memory allocations. Selecting the appropriate option may help you [improve app performance while profiling](#).
- » 4) Buttons to zoom in/out of the timeline.

3.4 Check Profile your app performance

Inspect your app's memory usage with Memory Profiler:

Memory Profiler overview

As indicated in figure 1, the default view for the Memory Profiler includes the following:

- » 6) The event timeline, which shows the activity states, user input events, and screen rotation events.
- » 7) The memory use timeline, which includes the following:
 - A stacked graph of how much memory is being used by each memory category, as indicated by the y-axis on the left and the color key at the top.
 - A dashed line indicates the number of allocated objects, as indicated by the y-axis on the right.
 - An icon for each garbage collection event.

However, if you're using a device running Android 7.1 or lower, not all profiling data is visible by default. If you see a message that says, "Advanced profiling is unavailable for the selected process," you need to [enable advanced profiling](#) to see the following:

- » Event timeline
- » Number of allocated objects
- » Garbage collection events

On Android 8.0 and higher, advanced profiling is always enabled for debuggable apps.

THANK YOU!

